ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP E1–E3

SO SÁNH CÁC MÔ HÌNH PHÂN LOẠI ẢNH TRÊN TẬP DỮ LIỆU MNIST

Học phần: Học sâu và ứng dụng trong thị giác máy tính

Mã học phần: CO5085

Học kỳ: HK261, năm học 2026–2027

Học viên: Ngô Duy Bảo

Mã số học viên: 2570395

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Mã nguồn: https://github.com/baond12/Deep_Learning_Course_HK261

BẢN DỰ THẢO PHỤC VỤ NHÓM REVIEW · 16/10/2026

MỤC LỤC

TÓM TẮT 3
CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG 4
CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN 6
CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH 9
CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH 15
CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN 21
KẾT LUẬN 24
TÀI LIỆU THAM KHẢO 25
PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN 26
PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI 31

TÓM TẮT

Báo cáo khảo sát các mô hình phân loại chữ số MNIST theo yêu cầu E1–E3 của học phần CO5085. E1 thiết lập baseline bằng Softmax, mạng truyền thẳng nhiều lớp (MLP) và mạng tích chập (CNN). E2 hiện thực cơ chế tự chú ý nhiều đầu (MSA) từ phép toán cơ bản và bằng mô-đun PyTorch, đối chiếu trên bốn cách lấy token. E3 xử lý bốn cách biểu diễn chuỗi bằng LSTM/GRU và dùng lại nguyên kết quả MLP/CNN E1 để so sánh. Ba bài cùng giải một nhiệm vụ bằng các kiến trúc và cách biểu diễn khác nhau.

Toàn bộ 19 cấu hình độc lập được huấn luyện 100 epoch, kích thước lô 256, trên cùng cách chia MNIST: 54.000 ảnh huấn luyện, 6.000 ảnh xác thực, 10.000 ảnh kiểm tra, seed 42; tổng cộng 1.900 bản ghi epoch. CNN E1 đạt accuracy test 99,27%; hai bản MSA dùng CNN-stem cùng đạt 99,28%; LSTM dùng CNN-stem ở E3 đạt 99,37% với 119.434 tham số. E2 vượt qua tám trường hợp kiểm chứng số học trên CPU/CUDA. E3 cho thấy độ nhạy khi thay thứ tự token trên validation, song chưa chứng minh khả năng ghi nhớ dài hạn. Các chênh lệch là quan sát từ một seed; không dùng để khẳng định ưu thế thống kê hoặc chọn lại cấu hình theo test [1]–[6].

Từ khóa: MNIST; phân loại ảnh; Softmax; MLP; CNN; multi-head self-attention; LSTM; GRU; biểu diễn token; ma trận nhầm lẫn.

CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG

1.1. Bài toán và mục tiêu đối chiếu

Cho ảnh $x \in \mathbb{R}^{1x28x28}$ biểu diễn một chữ số viết tay, nhiệm vụ là dự đoán nhãn $y \in \{0, 1, \ldots, 9\}$. Cả ba bài tập được tổ chức trên cùng một bài toán để tập trung vào khác biệt về biểu diễn đầu vào và kiến trúc: biến ảnh thành vector cho Softmax/MLP, giữ cấu trúc lân cận không gian cho CNN, chuyển ảnh thành token cho MSA và chuyển thành chuỗi cho LSTM/GRU. Yêu cầu của đề còn bao gồm vòng lặp huấn luyện tự viết, định lượng hiệu năng và phân tích lỗi, chứ không chỉ báo cáo độ chính xác cao nhất [1].

1.2. Dữ liệu và tiền xử lý

MNIST gồm ảnh xám 28 × 28 điểm ảnh thuộc 10 lớp. Bộ train gốc 60.000 ảnh được chia ngẫu nhiên với bộ sinh số khởi tạo seed = 42 thành 54.000 ảnh dùng cập nhật trọng số và 6.000 ảnh xác thực, tương ứng 90% và 10%; 10.000 ảnh test gốc được giữ riêng. Ảnh được chuyển thành tensor và chuẩn hóa theo $x' = (x - 0{,}1307)/0{,}3081$. Mã nguồn không thực hiện biến đổi tăng cường dữ liệu (augmentation) [2]–[4].

Bảng 1. Phân chia MNIST và vai trò trong đánh giá

Tập dữ liệu	Số lượng	Vai trò
Huấn luyện	54.000	Cập nhật trọng số qua lan truyền ngược
Xác thực	6.000	Theo dõi và chọn trạng thái tốt nhất
Kiểm tra	10.000	Đánh giá sau khi chọn mô hình

Checkpoint được chọn theo validation, không dựa vào test. Cùng hàm chia dữ liệu và seed 42 được sử dụng cho cả ba bài; hash chỉ số train/validation của E2 và E3 trùng nhau. E1 đặt seed trước vòng lặp mô hình; E2/E3 reset seed huấn luyện và generator bộ nạp cho từng cấu hình. Riêng E2 còn sao chép cùng toàn bộ trọng số ban đầu trong mỗi cặp MSA. Vì vậy cùng split không đồng nghĩa cùng khởi tạo hoặc quỹ đạo tối ưu. Mỗi cấu hình có một seed huấn luyện [2]–[5].

1.3. Quy trình huấn luyện và lựa chọn mô hình

Bảng 2. Cấu hình huấn luyện của 19 lượt thực nghiệm

Thành phần	Giá trị ghi nhận
Epoch	100 cho E1 E2 E3
Batch size	256 cho E1 E2 E3

Thành phần	Giá trị ghi nhận
Bộ tối ưu	Adam
Learning rate	0,001
Weight decay	0,0001
Hàm mất mát	CrossEntropyLoss (nhận logits)
Seed	42
Thiết bị	CUDA; E2/E3: Tesla P100-PCIE-16GB; E1 thiếu tên GPU
Lưu trạng thái	Sao chép trạng thái khi val accuracy tăng; nạp lại trước test

Ở mỗi mini-batch, chương trình xóa gradient, tính logits và CrossEntropyLoss, gọi backward() rồi optimizer.step(). Sau mỗi epoch, mô hình chuyển sang chế độ đánh giá để tính loss/accuracy trên validation. Trạng thái tốt nhất được sao chép khi validation accuracy tăng; nếu bằng nhau giữ epoch sớm nhất. Cả 19 cấu hình chạy đủ 100 epoch, không dùng dừng sớm, scheduler hay gradient clipping. Trước khi test, chương trình nạp lại trạng thái tốt nhất [2]–[4].

CrossEntropyLoss xử lý phần log-softmax ổn định về số học bên trong, vì vậy các model trả về logits, không đặt thêm softmax trước hàm mất mát. Khi suy luận, argmax(logits) và argmax(softmax(logits)) cho cùng nhãn dự đoán. Thời gian "training_time" bao gồm các lượt huấn luyện và xác thực, cùng thao tác lưu/nạp trạng thái tốt nhất trong hàm fit_classifier; không bao gồm đánh giá test hoặc tạo hình [2].

1.4. Chỉ số đánh giá và cách đọc hình

Độ chính xác (accuracy) là số dự đoán đúng chia tổng số ảnh. F1 macro (F1 trung bình theo lớp, thang 0–1) là trung bình cộng F1 của từng lớp, vì vậy phản ánh hiệu năng theo lớp mà không chỉ dựa vào các lớp nhiều mẫu. Bên cạnh đó, báo cáo xét test loss, số tham số có requires_grad, thời gian huấn luyện và đường cong train/validation. Ma trận nhầm lẫn đặt nhãn thật theo hàng, nhãn dự đoán theo cột; các ô ngoài đường chéo cho biết dạng nhầm lẫn. Hình lỗi là tối đa 25 mẫu sai xuất hiện đầu tiên khi duyệt test, không phải lấy mẫu ngẫu nhiên [2], [5], [6]. Accuracy được biểu thị theo %, còn mức chênh giữa hai accuracy dùng đơn vị điểm phần trăm (đpt).

Lưu ý khi đọc đường cong: độ chính xác huấn luyện được đo ở chế độ model.train(), còn xác thực ở model.eval(). Vì dropout chỉ hoạt động trong huấn luyện, có thể xuất hiện val accuracy lớn hơn train accuracy mà không tự nó chứng minh rò rỉ dữ liệu. Các nhận xét về dấu hiệu quá khớp chỉ dựa trên diễn biến loss và chênh lệch giữa hai tập trong lần chạy đã ghi nhận.

CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN

Câu hỏi E1: với cùng dữ liệu và quy trình huấn luyện, việc tăng khả năng biểu diễn từ mô hình tuyến tính sang MLP rồi CNN gắn với thay đổi nào về độ chính xác, số tham số và dạng lỗi? Ba kiến trúc dưới đây được huấn luyện độc lập để thiết lập mốc so sánh cho E2 và E3 [1], [2].

2.1. Câu hỏi và thiết kế ba kiến trúc

Softmax Regression được xây dựng bằng Flatten rồi Linear(784, 10): $z = Wx + b$, với $x \in \mathbb{R}^{784}$ và $W \in \mathbb{R}^{10x784}$. Mô hình có $784 \times 10 + 10 = 7.850$ tham số. Đây là bộ phân loại tuyến tính trên không gian điểm ảnh gốc, không có các tầng học đặc trưng phi tuyến; đầu ra là logits để đưa vào CrossEntropyLoss [2], [7].

MLP nhận vector 784 chiều, đi qua Linear(784, 256) → ReLU → Dropout(0,2) → Linear(256, 128) → ReLU → Dropout(0,2) → Linear(128, 10). Số tham số lần lượt của ba tầng tuyến tính là 200.960, 32.896 và 1.290; tổng cộng 235.146. Nhờ ReLU, MLP xây dựng được ranh giới quyết định phi tuyến, nhưng phép làm phẳng khiến kiến trúc không khai thác trực tiếp quan hệ láng giềng giữa các pixel [2], [7].

CNN gồm hai khối Conv2d(1→32, 3×3, padding 1) → ReLU → MaxPool2d(2) và Conv2d(32→64, 3×3, padding 1) → ReLU → MaxPool2d(2). Kích thước tensor lần lượt là 1×28×28 → 32×14×14 → 64×7×7; sau Flatten, head Linear(3136, 128) → ReLU → Dropout(0,3) → Linear(128, 10). Bốn tầng có tham số 320, 18.496, 401.536 và 1.290, tổng 421.642. Tích chập chia sẻ trọng số giữa các vị trí và tổng hợp đặc trưng cục bộ, phù hợp với cấu trúc nét viết của MNIST [2], [8].

2.2. Kết quả định lượng

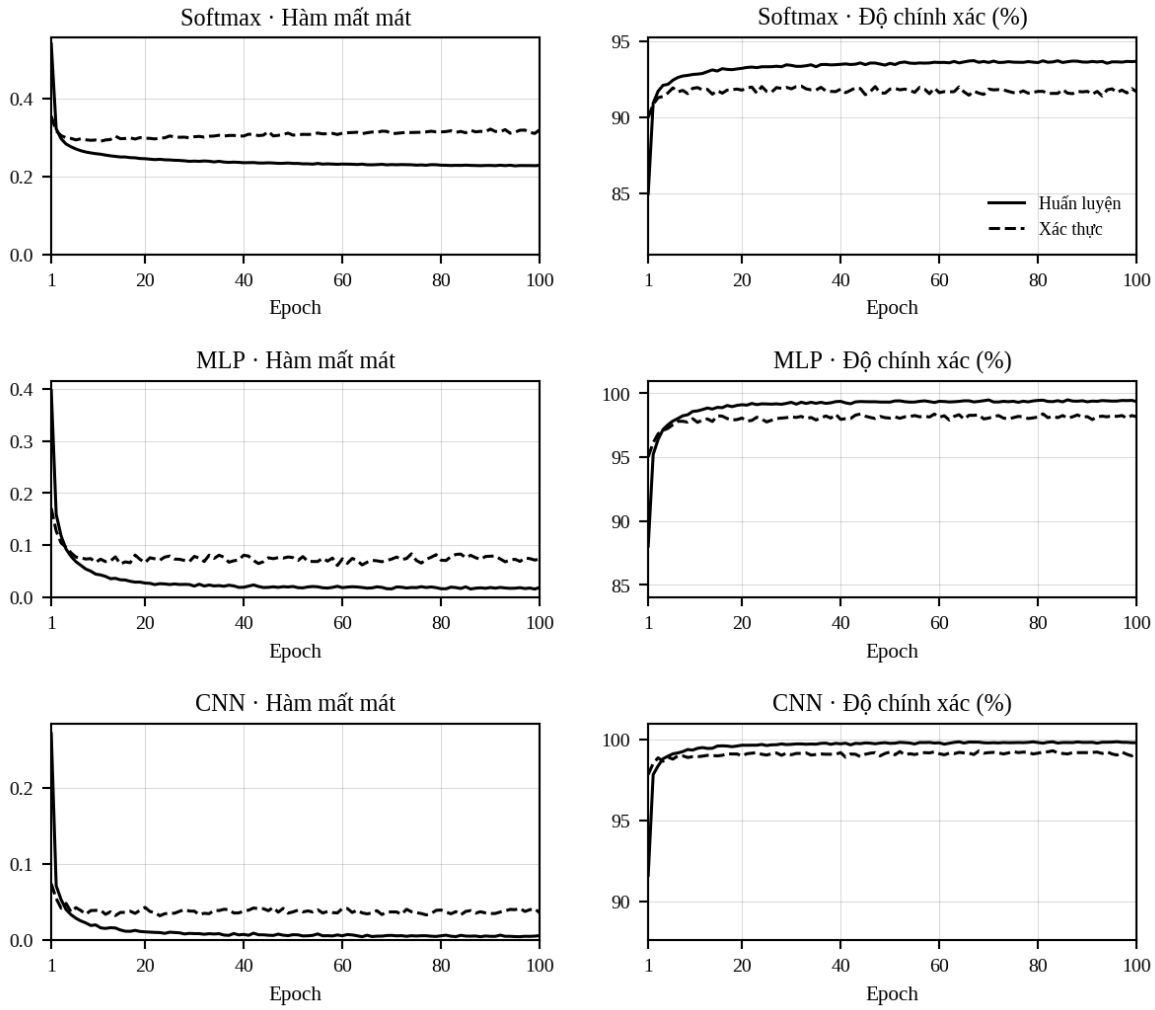
Bảng 3. Kết quả E1 (100 epoch, batch 256)

Mô hình	Val tốt nhất	Test accuracy	Macro-F1	Tham số	Epoch tốt nhất
Softmax	92,03%	92,59%	0,924782	7.850	32
MLP	98,40%	98,63%	0,986219	235.146	81
CNN	99,32%	99,27%	0,992631	421.642	83

CNN đạt test accuracy 99,27%, cao hơn MLP 0,64 điểm phần trăm và Softmax 6,68 điểm phần trăm; MLP đạt 98,63%, cao hơn Softmax 6,04 điểm phần trăm. F1 macro lần lượt là 0,924782, 0,986219 và 0,992631 cho Softmax, MLP, CNN. Thời gian train + validation tương ứng 718,3; 760,6; 770,0 giây. So với bộ E1 cũ (20 epoch, lô 16), accuracy tăng 0,50; 0,81; 0,10 điểm phần trăm. Vì thay đổi đồng thời epoch và batch

size, đồng thời quá trình tiêu thụ RNG thay đổi, chưa thể quy cải thiện riêng cho một yếu tố hoặc khẳng định cấu hình đã tối ưu [5], [6].

2.3. Diễn biến hội tụ

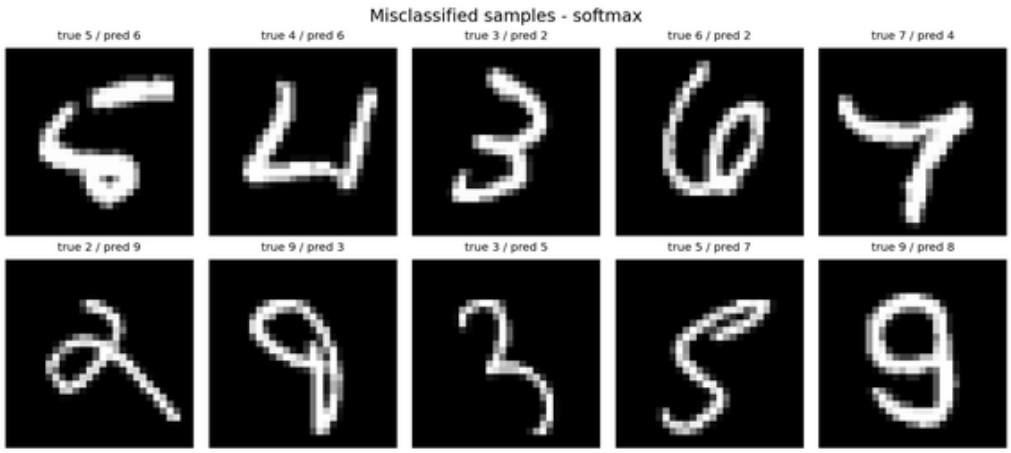


Hình 1. Diễn biến loss và accuracy của ba mô hình E1. Nét liền: huấn luyện; nét đứt: xác thực. Dựng lại từ lịch sử 08 [6].

Softmax giảm training loss từ 0,539 xuống 0,228; validation accuracy tốt nhất đạt 92,03% tại epoch 32, còn epoch 100 đạt 91,65%. Đây là dấu hiệu validation không tiếp tục cải thiện ổn định trong cấu hình đã chạy, chưa chứng minh giới hạn tối ưu của mô hình. MLP đạt best validation accuracy 98,40% tại epoch 81; training loss cuối là 0,0189, trong khi validation loss khoảng 0,0733. CNN đạt best validation accuracy 99,32% tại epoch 83; training loss cuối khoảng 0,0058 và validation loss 0,0357. Khoảng cách train–validation vẫn tồn tại; checkpoint để test là các epoch 32, 81 và 83, không mặc định epoch 100 [5], [6].

2.4. Phân tích lỗi

Trên 10.000 ảnh test, Softmax sai 741 ảnh, MLP sai 137 ảnh và CNN sai 73 ảnh. Ma trận E1 mới (Phụ lục A) ghi nhận Softmax nhầm 5 thành 8 ở 37 ảnh, 5 thành 3 ở 36 ảnh và 2 thành 8 ở 35 ảnh. CNN giảm mạnh số lỗi, nhưng vẫn có 8 ảnh số 9 bị nhận thành 4 và 6 ảnh số 5 thành 3. Đây là thống kê của checkpoint đã chọn bằng validation; chưa đủ để xác định nguyên nhân từng lỗi [5].



Hình 2. Mười mẫu sai đầu tiên của Softmax trong lần chạy E1 mới. “true/pred” tương ứng nhãn thật/nhãn dự đoán [2], [5].

Các mẫu lỗi mới minh họa những nét viết dễ gây nhầm: số 5 thành 6, số 4 thành 6, số 3 thành 2 và số 7 thành 4. Hình chỉ lấy mười lỗi đầu từ danh sách tối đa 25 lỗi theo thứ tự test, không lấy mẫu ngẫu nhiên. Vì vậy hình dùng để minh họa định tính, không ước lượng được tỷ lệ lỗi theo kiểu nét viết hoặc chứng minh chắc chắn cơ chế gây sai [2], [5].

CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH

Câu hỏi E2 gồm hai phần: bản tự hiện thực và mô-đun PyTorch có cho kết quả số học tương đương khi dùng cùng trọng số hay không; và bốn cách tạo token cho kết quả phân loại, số tham số, thời gian ghi nhận khác nhau thế nào? Kiểm chứng số học được thực hiện riêng trước khi phân tích các lượt huấn luyện [1], [3].

3.1. Câu hỏi thực nghiệm và bốn cách lấy token

E2 đối chiếu MSA tự viết với nn.MultiheadAttention trong từng cách lấy token, đồng thời so sánh patch, hàng ảnh, CNN-stem và patch chồng lấn. Bộ kết quả mới gồm tám cấu hình, chạy 100 epoch, lô 256. Trong nhóm pixel/hàng của đề, chọn mỗi hàng là một token; trong nhóm biến thể, chọn patch chồng lấn. Ảnh đầu vào, cách chia dữ liệu và chuẩn hóa giữ như E1 [1], [3], [5].

Patch rời dùng Conv2d kernel 7, stride 7 tạo lưới 4×4; hàng ảnh dùng Linear(28,64). CNN-stem gồm Conv(1→32,3×3,padding 1) → ReLU → Pool2 → Conv(32→64,3×3,padding 1) → ReLU → Pool2, tạo bản đồ 64×7×7; mỗi vị trí không gian là một token. Patch chồng lấn dùng kernel 7, stride 3, không padding, tạo lưới 8×8 và phủ hết ảnh 28×28. Cả bốn thêm CLS và nhúng vị trí học được, rồi dùng LayerNorm và Linear(64,10) trên CLS để phân loại [3], [9].

Bảng 4. Token và số tham số của mỗi bản MSA

Thành phần	Patch	Hàng	CNN-stem	Chồng lấn
Token ảnh	16	28	49	64
Tổng có CLS	17	29	50	65
Tham số mỗi bản	38.602	38.026	56.330	41.674
Embedding / head	64 / 4	64 / 4	64 / 4	64 / 4
FFN / số block	128 / 1	128 / 1	128 / 1	128 / 1

3.2. Công thức và cơ chế MSA

Với X có dạng B×N×64, phép chiếu tuyến tính tạo Q, K, V rồi chia thành bốn head, mỗi head rộng 16. Ở head i, $A_i = \text{softmax}(Q_iK_i^T/\sqrt{16})$, $H_i = \text{Dropout}(A_i)V_i$; các H_i được ghép và chiếu về B×N×64. Bản tự viết dùng Linear, reshape/permute, nhân ma trận và softmax; không gọi nn.MultiheadAttention hoặc nn.TransformerEncoder. Bản PyTorch gọi nn.MultiheadAttention với batch_first=True và need_weights=False [3], [10].

Encoder chung có một block tiền chuẩn hóa: $X_1 = X + \text{Dropout}(\text{MSA}(\text{LayerNorm}(X)))$; $X_2 = X_1 + \text{FFN}(\text{LayerNorm}(X_1))$. FFN rộng 64→128→64, dùng GELU; dropout có xác suất 0,1. Cả hai bản dùng cùng vị trí dropout, gồm dropout attention và nhánh residual. Với CLS, số phần tử ma trận attention mỗi head lần lượt là 289, 841, 2.500 và 4.225. Đây là chi phí riêng phần attention, không đồng nhất với thời gian toàn bộ mạng [3], [10].

3.3. Kiểm chứng MSA và kiểm soát thực nghiệm

Trước khi huấn luyện, mỗi cặp được sao chép toàn bộ trọng số, gồm tokenizer, CLS, nhúng vị trí, LayerNorm, FFN, head và QKV/output projection. Kiểm tra dùng đầu vào tổng hợp, đối chiếu đầu ra attention, gradient đầu vào và tham số; ở classifier còn xét logits suy luận, logits/loss khi train nhưng tắt dropout, cùng gradient toàn bộ tham số. CPU float64 kiểm tra thêm một bước Adam; CUDA float32 kiểm tra đầu ra và gradient trên thiết bị huấn luyện [3], [5].

Bảng 5. Kiểm chứng tương đương tại cùng trọng số

Tokenizer	CPU float64	CUDA float32	Kết quả
Patch	7,11E-15	1,91E-06	PASS / PASS
Hàng	3,55E-15	2,86E-06	PASS / PASS
CNN-stem	7,11E-15	3,81E-06	PASS / PASS
Chồng lấn	8,88E-15	3,81E-06	PASS / PASS

Cả tám trường hợp đều PASS. Ngưỡng CPU là atol = 10^{-10}, rtol = 10^{-8}; CUDA là atol = 2×10^{-5}, rtol = 2×10^{-4}. Sai số bảng là cực đại tuyệt đối trong các mục đã ghi, không thay thế điều kiện so sánh kết hợp atol/rtol. Mỗi tokenizer kiểm tra 20 tensor tham số, riêng CNN-stem là 22. Đây là bằng chứng số học hữu hạn, không phải chứng minh cho mọi đầu vào [3], [5].

Khi train thật, hai mô hình trong mỗi cặp có cùng hash khởi tạo; seed huấn luyện và generator của bộ nạp dữ liệu được reset để có cùng thứ tự batch. Dropout cùng xác suất và vị trí nhưng không ép mặt nạ trùng nhau giữa các kernel, nên quỹ đạo tối ưu có thể khác. Bộ kết quả ghi nhận Tesla P100-PCIE-16GB, PyTorch 2.2.1+cu118 và CUDA 11.8; hash nguồn và split đã được đối chiếu. Checkpoint được chọn theo validation, test không dùng để chọn lại cấu hình [3], [5].

3.4. Kết quả và hai phép so sánh

Bảng 6. Kết quả E2 với 100 epoch, kích thước lô 256

Cấu hình	Val tốt nhất	Test accuracy	F1 macro	Tham số	Train + val (s)	Epoch tốt nhất
Tự viết · Patch	97,48%	97,59%	0,975703	38.602	738,8	98
PyTorch · Patch	97,13%	97,05%	0,970280	38.602	733,0	55
Tự viết · Hàng	98,05%	98,04%	0,980230	38.026	750,6	83
PyTorch · Hàng	98,15%	98,24%	0,982280	38.026	751,3	97
Tự viết · CNN-stem	99,32%	99,28%	0,992767	56.330	763,6	82
PyTorch · CNN-stem	99,28%	99,28%	0,992781	56.330	827,2	59
Tự viết · Chồng lấn	98,47%	98,53%	0,985199	41.674	739,7	97
PyTorch · Chồng lấn	98,45%	98,33%	0,983174	41.674	737,4	82

CNN-stem dẫn đầu trong từng bản MSA, cùng đạt test accuracy 99,28% và sai 72 ảnh; F1 macro lần lượt 0,992767 và 0,992781. Patch chồng lấn đạt 98,53%/98,33%, hàng ảnh 98,04%/98,24%, patch rời 97,59%/97,05% (tự viết/PyTorch). Cùng số ảnh đúng không có nghĩa hai mô hình sai đúng cùng các ảnh [5].

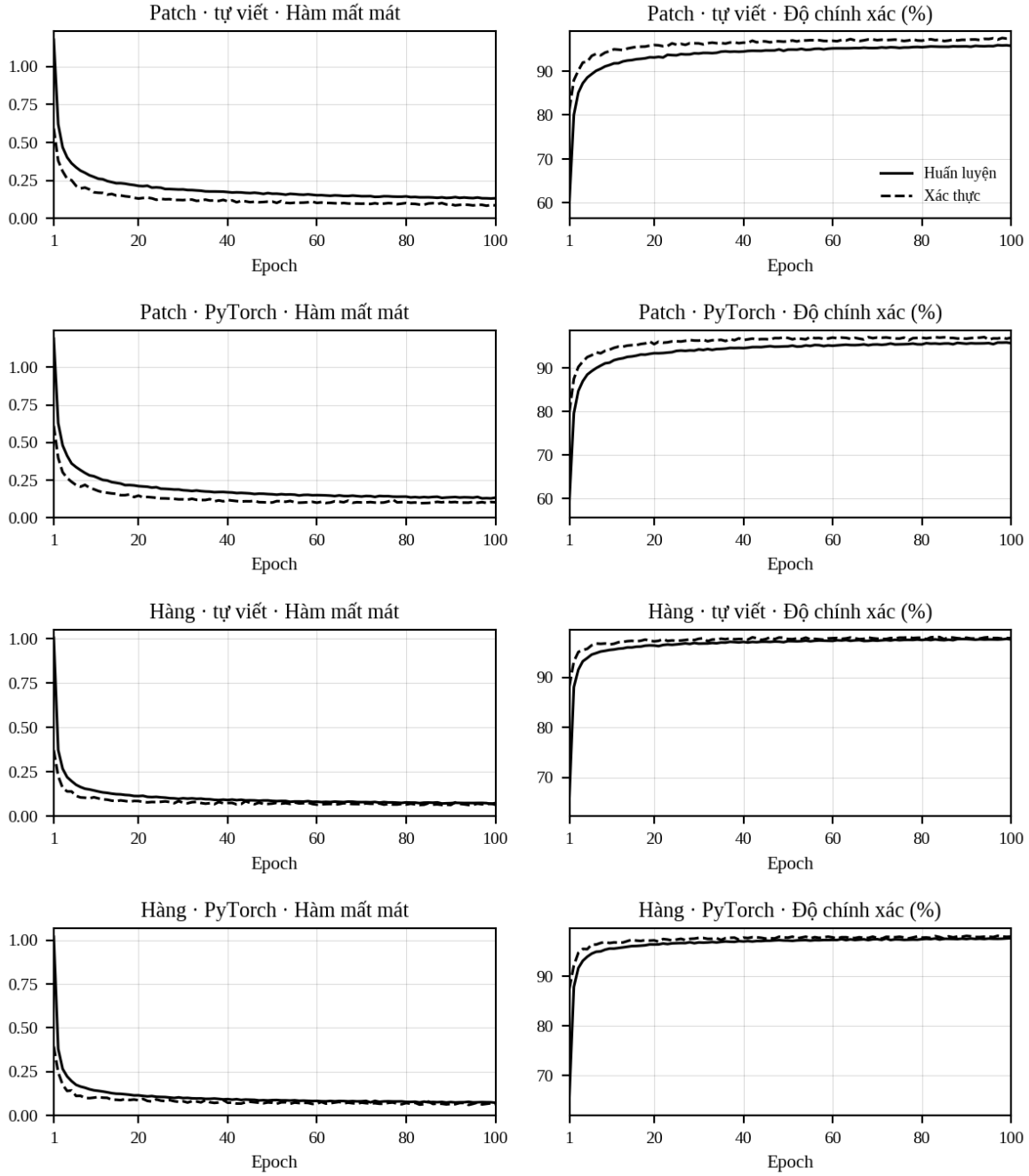
Bảng 7. PyTorch so với tự viết trong từng tokenizer

Tokenizer	Δ accuracy (đpt)	Tự viết (s)	PyTorch (s)	Tỷ số thời gian
Patch	-0,54	738,8	733,0	0,9922
Hàng	0,20	750,6	751,3	1,0010
CNN-stem	0,00	763,6	827,2	1,0834
Chồng lấn	-0,20	739,7	737,4	0,9969

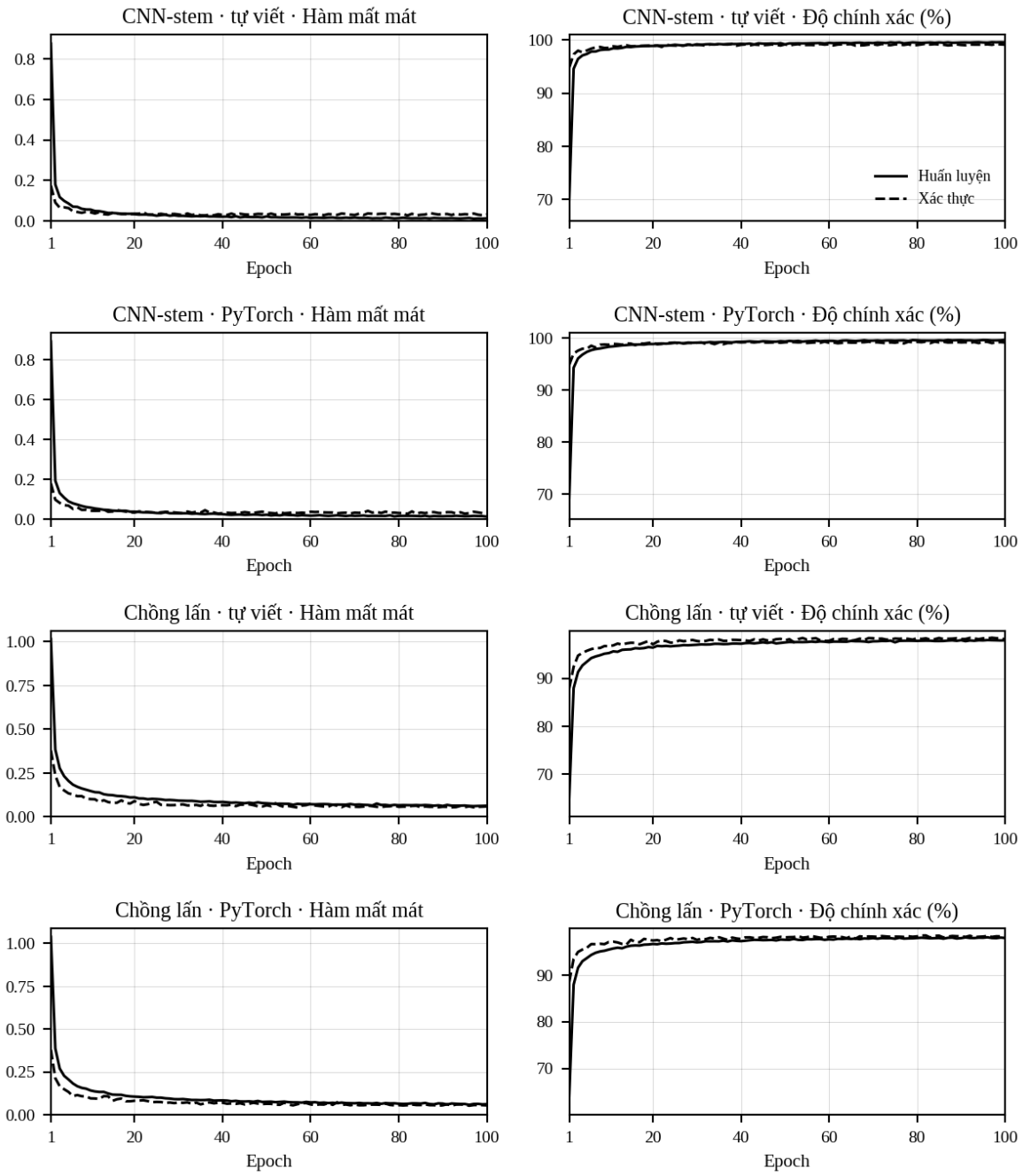
Chênh accuracy PyTorch trừ tự viết nằm từ −0,54 đến +0,20 điểm phần trăm, không theo một chiều cố định. Với CNN-stem, thời gian PyTorch cao hơn khoảng 8,34%; ba cặp còn lại có tỷ số gần 1 trong lượt đo này. Không từ đó kết luận một hiện thực luôn nhanh hoặc chính xác hơn. Kiểm chứng số học và so sánh huấn luyện trả lời hai câu hỏi khác nhau [3], [5].

Trong cùng bản MSA, CNN-stem hơn patch rời 1,69 điểm phần trăm ở bản tự viết và 2,23 ở PyTorch; patch chồng lấn hơn patch rời 0,94 và 1,28 điểm phần trăm. CNN-stem đồng thời bổ sung CNN và tăng capacity; overlap thay đổi số token cùng nhúng vị trí. Vì vậy đây là so sánh các cấu hình biểu diễn, chưa tách được tác động nhân quả của từng yếu tố. Đo thời gian một lượt cũng chưa đủ suy ra quy luật tỷ lệ theo số token [3], [5].

3.5. Đường cong huấn luyện và xác thực



Hình 3. Loss và accuracy E2 của patch rời và hàng ảnh, gồm bản tự viết/PyTorch. Nét liền: huấn luyện; nét đứt: xác thực. Dựng từ lịch sử 08 [6].

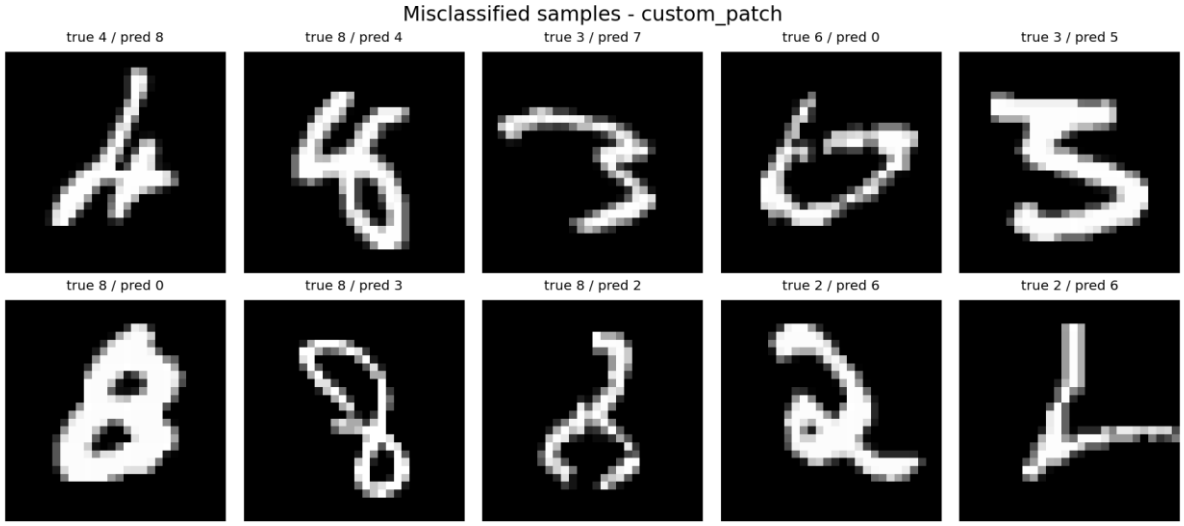


Hình 4. Loss và accuracy E2 của CNN-stem và patch chồng lấn. Quy ước nét như Hình 3; nguồn: lịch sử 08 [6].

Cả tám lượt đều giảm loss mạnh ban đầu. Đến epoch 100, training loss của hai bản CNN-stem khoảng 0,0133/0,0116, validation loss khoảng 0,0325/0,0331; khoảng cách này gợi ý mức quá khớp trong lượt chạy. Với patch rời, train accuracy cuối khoảng 95,7%, thấp hơn validation; cần xét khác biệt dropout giữa train/eval như Chương 1, không coi đó là bằng chứng rò rỉ dữ liệu. Best epoch lần lượt là 98/55 (patch), 83/97 (hàng), 82/59 (CNN-stem), 97/82 (overlap), nên test không mặc định dùng epoch 100 [5], [6].

3.6. Phân tích lỗi và giới hạn diễn giải

Từ dự đoán trên 10.000 ảnh test, số lỗi tự viết/PyTorch lần lượt là 241/295 (patch), 196/176 (hàng), 72/72 (CNN-stem), 147/167 (overlap). Ở custom_patch, nhầm 7→2 và 3→5 đều 15 ảnh; custom_row nhầm 3→5 ở 15 ảnh. CNN-stem tự viết còn 8 lỗi 4→9, còn bản PyTorch có 5 lỗi 9→4 và 5 lỗi 7→1. Ma trận của đủ tám mô hình được trình bày ở Phụ lục A [5].



Hình 5. Mười lỗi đầu của custom_patch mới theo thứ tự test; true/pred là nhãn thật/dự đoán. Hình minh họa định tính, không lấy mẫu ngẫu nhiên [5].

Các mẫu cho thấy một số nét viết dễ gây nhầm giữa chữ số, nhưng chưa xác định được nguyên nhân từng dự đoán. Không có bản đồ attention theo ảnh để quy lỗi cho một head cụ thể. Một seed, một bộ dữ liệu và số liệu thời gian chưa kiểm soát tải cạnh tranh giới hạn việc khái quát kết luận [3], [5].

CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH

Câu hỏi E3: trên từng cách biểu diễn ảnh thành chuỗi, LSTM và GRU khác nhau thế nào về accuracy, số tham số, lỗi dự đoán và độ nhạy thứ tự token? Hai mốc MLP/CNN được lấy nguyên từ E1, không thêm lượt huấn luyện baseline [1], [4], [5].

4.1. Biểu diễn ảnh và liên hệ với E1, E2

E3 dùng cùng MNIST và split như E1/E2, chuyển ảnh thành chuỗi theo bốn cách. Hàng ảnh và patch giữ pixel thô; CNN-stem học đặc trưng không gian rồi quét từng vị trí; patch chồng lấn tăng số bước bằng các cửa sổ giao nhau. Không thêm CLS hoặc nhúng vị trí. Thứ tự quét là từ trái sang phải rồi từ trên xuống dưới; với hàng ảnh là từ trên xuống dưới [4].

Bảng 8. Độ dài và chiều đặc trưng chuỗi E3

Biểu diễn	Số bước T	Chiều D	Đặc trưng đầu vào
Hàng ảnh	28	28	28 pixel mỗi hàng
Patch rời	16	49	7×7 pixel, stride 7
CNN-stem	49	64	Vị trí feature map 7×7
Patch chồng lấn	64	49	7×7 pixel, stride 3

Patch rời dùng kernel 7, stride 7; patch chồng lấn dùng kernel 7, stride 3, không padding và phủ đủ biên ảnh 28×28. CNN-stem gồm Conv(1→32,3,pad1) → ReLU → Pool2 → Conv(32→64,3,pad1) → ReLU → Pool2, tạo bản đồ 64×7×7. Khác E2, hàng/patch E3 không dùng projection lên 64 chiều; E2 còn có token CLS và nhúng vị trí, còn E3 dùng trạng thái ẩn cuối h_n[-1] để phân loại; do đó không thể diễn giải so sánh E2/E3 là chỉ thay attention bằng RNN [3], [4].

4.2. Kiến trúc mạng hồi quy và cách đọc trạng thái cuối

Mỗi mô hình dùng nn.LSTM hoặc nn.GRU một tầng, một chiều, hidden size H = 128. Trạng thái ẩn cuối của lớp cuối h_n[-1] đi qua Dropout(0,1) và Linear(128,10) để tạo logits. Với một tầng, dropout nội bộ RNN bằng 0; dropout chỉ nằm ở head. MLP/CNN không được tạo mới hoặc huấn luyện lại: chương trình đọc nguyên metrics/history E1 và kiểm tra cấu hình trước khi chạy tám RNN [2], [4].

LSTM duy trì trạng thái ô nhớ c_t và trạng thái ẩn h_t: $c_t = f_t \odot c_{t-1} + i_t \odot g_t$; $h_t = o_t \odot \tanh(c_t)$. GRU dùng cổng cập nhật và đặt lại để kết hợp trạng thái trước với trạng thái ứng viên, không có ô nhớ tách biệt. Hai kiến trúc khác nhau nên không đặt yêu cầu LSTM/GRU phải khớp đầu ra như hai hiện thực cùng MSA của E2 [4], [11].

Với hai vector bias theo cách đếm PyTorch, khối LSTM có 4H(D+H+2) tham số, GRU có 3H(D+H+2); head thêm 1.290. Hàng ảnh cho 82.186/61.962 tham số (LSTM/GRU); patch rời và chồng lấn cùng cho 92.938/70.026. CNN-stem thêm 18.816 tham số tích chập và dùng D = 64, tổng 119.434/94.602. GRU ít tham số hơn trong cùng biểu diễn, nhưng tốc độ còn phụ thuộc kernel và thiết bị [4].

4.3. Kết quả và đối chiếu mốc E1

Bảng 9. Tám mô hình hồi quy E3 (100 epoch, batch 256)

Mô hình	Val tốt nhất	Accuracy test	F1 macro	Tham số	Train + val (s)	Best epoch
LSTM · Hàng ảnh	98,98%	98,93%	0,989242	82.186	723,7	95
GRU · Hàng ảnh	99,02%	98,94%	0,989358	61.962	699,9	78
LSTM · Patch rời	98,28%	98,20%	0,981864	92.938	703,9	93
GRU · Patch rời	98,18%	98,36%	0,983462	70.026	688,0	76
LSTM · CNN-stem	99,40%	99,37%	0,993709	119.434	743,1	58
GRU · CNN-stem	99,45%	99,27%	0,992663	94.602	756,5	89
LSTM · Patch chồng lấn	99,13%	98,98%	0,989735	92.938	755,0	47
GRU · Patch chồng lấn	98,93%	99,12%	0,991074	70.026	749,3	94

Bảng 10. Hai baseline dùng lại nguyên kết quả E1

Nguồn E1	Val tốt nhất	Accuracy test	F1 macro	Tham số	Train + val (s)	Best epoch
MLP (tái sử dụng)	98,40%	98,63%	0,986219	235.146	760,6	81
CNN (tái sử dụng)	99,32%	99,27%	0,992631	421.642	770,0	83

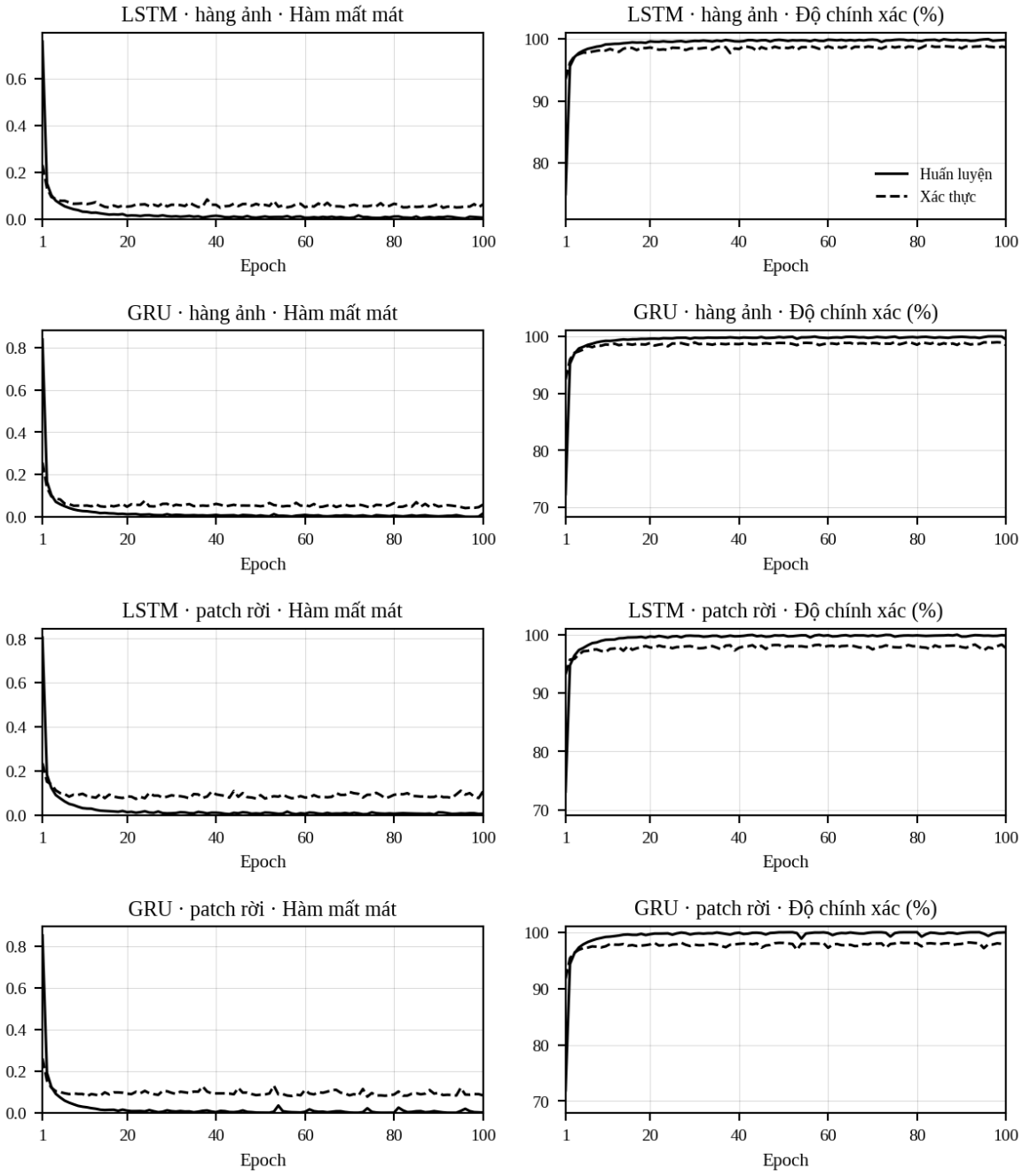
LSTM CNN-stem đạt 99,37%, hơn CNN E1 0,10 điểm phần trăm và MLP E1 0,74 điểm phần trăm; số tham số bằng khoảng 28,33% CNN E1. GRU CNN-stem đạt 99,27%, bằng accuracy CNN E1 nhưng F1 macro khác nhau. Cùng accuracy không chứng minh cùng tập lỗi. Các số E1 là kết quả đã có, không phải hai lượt train hoặc test mới của E3 [5].

Bảng 11. GRU trừ LSTM trên cùng cách biểu diễn

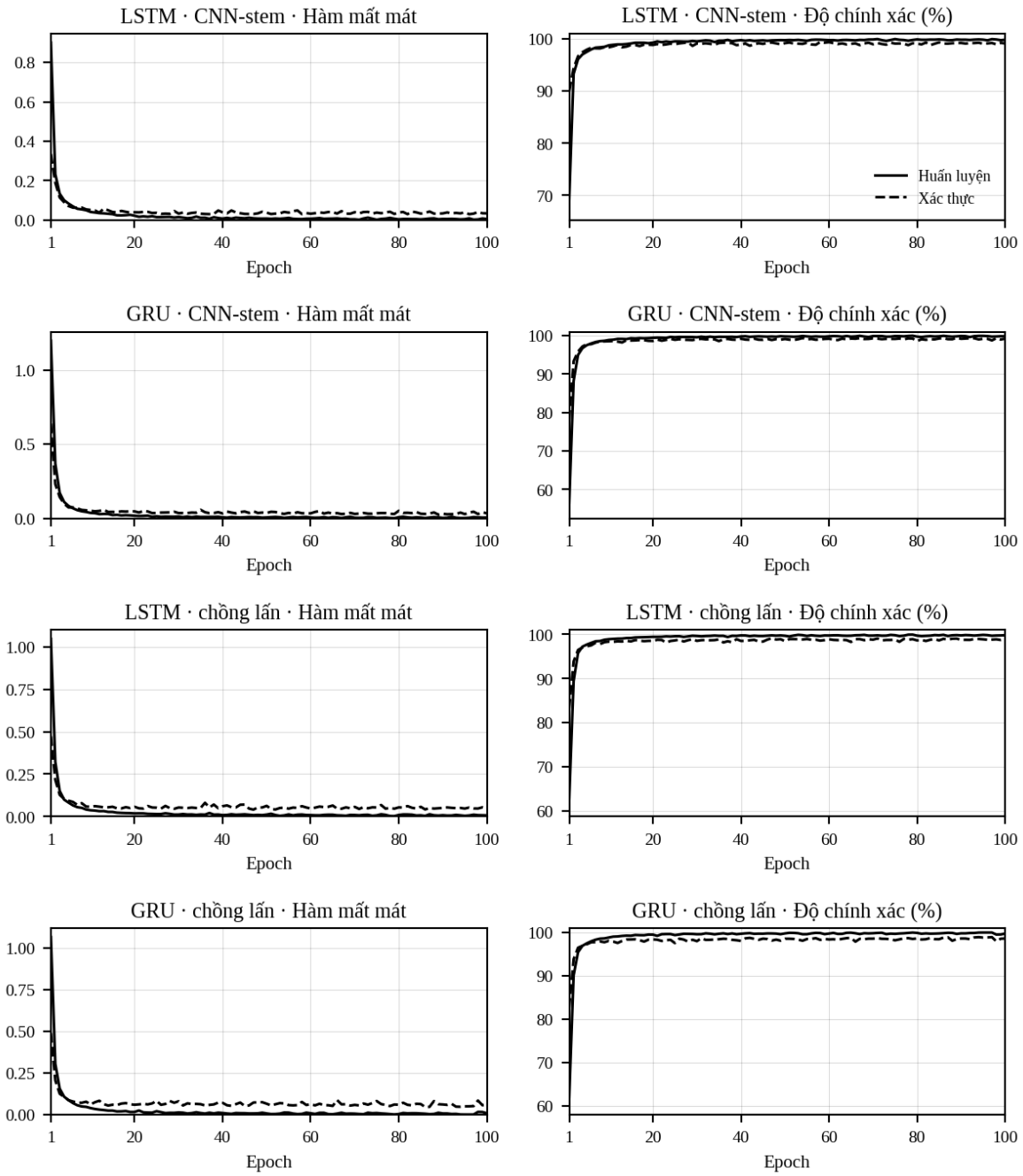
Biểu diễn	Δ accuracy (đpt)	Tham số LSTM	Tham số GRU	Δ thời gian (s)
Hàng ảnh	0,01	82.186	61.962	-23,8
Patch rời	0,16	92.938	70.026	-15,9
CNN-stem	-0,10	119.434	94.602	13,4
Patch chồng lấn	0,14	92.938	70.026	-5,8

GRU hơn LSTM 0,01; 0,16; 0,14 điểm phần trăm tương ứng hàng ảnh, patch rời, patch chồng lấn; với CNN-stem, LSTM hơn GRU 0,10 điểm phần trăm. GRU ít tham số hơn ở cả bốn cặp, nhưng thời gian ngắn hơn ở ba cặp và dài hơn ở CNN-stem. Bảng chỉ mô tả một lượt; không khẳng định GRU hoặc LSTM luôn tốt hơn. CNN-stem tốt nhất trong từng họ, còn patch chồng lấn hơn patch rời 0,78/0,76 điểm phần trăm (LSTM/GRU) [5].

4.4. Diễn biến huấn luyện và xác thực



Hình 6. Loss và accuracy E3 của hàng ảnh và patch rời, với LSTM/GRU. Nét liền: huấn luyện; nét đứt: xác thực. Nguồn: lịch sử 08 [6].

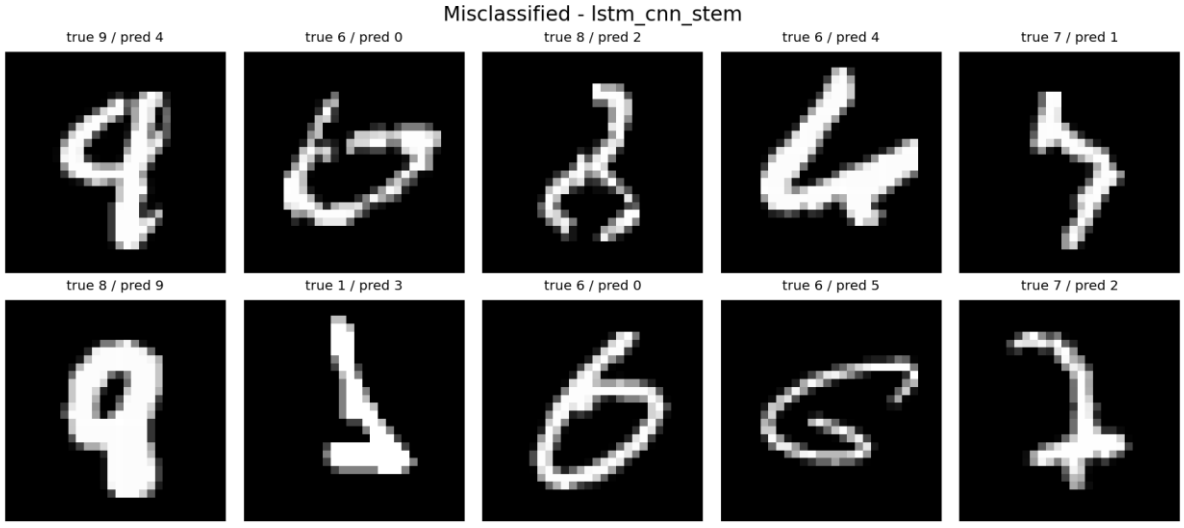


Hình 7. Loss và accuracy E3 của CNN-stem và patch chồng lấn. Quy ước nét như Hình 6; nguồn: lịch sử 08 [6].

Các lượt giảm loss mạnh ở giai đoạn đầu nhưng validation vẫn dao động về cuối. Epoch 100, LSTM patch có train loss 0,0052 và validation loss 0,1073; GRU patch tương ứng 0,0016 và 0,0837, cho thấy khoảng cách khái quát hóa trong lượt chạy. Với CNN-stem, validation loss cuối khoảng 0,0344/0,0331 (LSTM/GRU). Best epoch của các cặp hàng, patch, CNN-stem, chồng lấn lần lượt là 95/78, 93/76, 58/89, 47/94, nên checkpoint để test không mặc định là epoch 100 [5], [6].

4.5. Phân tích lỗi và độ nhạy thứ tự

Trên 10.000 ảnh test, số lỗi LSTM/GRU lần lượt là 107/106 (hàng), 180/164 (patch), 63/73 (CNN-stem), 102/88 (chồng lấn). LSTM hàng nhầm 5→3 ở 11 ảnh và 4→9 ở 10 ảnh; LSTM CNN-stem còn 7 lỗi 4→9, GRU CNN-stem còn 7 lỗi 9→4. Ma trận của đủ tám mô hình nằm ở Phụ lục A. Các thống kê xác nhận dạng nhầm lẫn, chưa xác định nguyên nhân từng lỗi [5].



Hình 8. Mười mẫu sai đầu của LSTM CNN-stem theo thứ tự test, trích từ tối đa 25 lỗi lưu bởi pipeline. true/pred là nhãn thật/dự đoán; hình không lấy mẫu ngẫu nhiên [4], [5].

Bảng 12. Accuracy validation khi giữ và thay thứ tự token

Mô hình	Thứ tự gốc (%)	Đảo ngược (%)	Xáo cố định (%)
LSTM · Hàng ảnh	98,98	40,32	20,40
GRU · Hàng ảnh	99,02	39,02	17,97
LSTM · Patch rời	98,28	18,00	30,05
GRU · Patch rời	98,18	13,20	31,85
LSTM · CNN-stem	99,40	23,20	31,20
GRU · CNN-stem	99,45	19,43	32,40
LSTM · Patch chồng lấn	99,13	15,53	25,07
GRU · Patch chồng lấn	98,93	12,32	15,23

Order probe giữ nguyên checkpoint đã chọn, biến ảnh thành token rồi đảo toàn bộ thứ tự hoặc dùng hoán vị cố định seed 42. Không huấn luyện thêm và không dùng test. Accuracy validation giảm từ khoảng 98,18–99,45% xuống 12,32–40,32% khi đảo và 15,23–32,40% khi xáo. Chuỗi raster theo không gian ảnh có ảnh hưởng rõ trong kiểm tra này [4], [5].

Tuy nhiên, thứ tự mới khác phân phối lúc train. Với CNN-stem, đặc trưng đã chứa ngữ cảnh cục bộ trước khi bị đảo; với patch chồng lấn, các token chia sẻ pixel. Vì vậy phép thử chỉ phản ánh độ nhạy thứ tự không gian của mô hình đã học, không tách được trí nhớ dài hạn và không chứng minh quan hệ thời gian thực. MLP/CNN E1 làm baseline chất lượng/chi phí, không nhận cùng thao tác đảo token này.

CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN

5.1. Đối chiếu 19 cấu hình độc lập

Bảng 13. Chỉ số test của 19 cấu hình độc lập

Bài	Mã thí nghiệm	Accuracy	Macro-F1	Test loss
E1	softmax	92,59%	0,924782	0,27112
E1	mlp	98,63%	0,986219	0,06062
E1	cnn	99,27%	0,992631	0,02589
E2	custom_patch	97,59%	0,975703	0,07868
E2	pytorch_patch	97,05%	0,970280	0,09693
E2	custom_row	98,04%	0,980230	0,06553
E2	pytorch_row	98,24%	0,982280	0,05759
E2	custom_cnn_stem	99,28%	0,992767	0,02449
E2	pytorch_cnn_stem	99,28%	0,992781	0,02404
E2	custom_overlap_patch	98,53%	0,985199	0,05002
E2	pytorch_overlap_patch	98,33%	0,983174	0,05610
E3	lstm_row	98,93%	0,989242	0,04329
E3	gru_row	98,94%	0,989358	0,04495
E3	lstm_patch	98,20%	0,981864	0,06683
E3	gru_patch	98,36%	0,983462	0,06961
E3	lstm_cnn_stem	99,37%	0,993709	0,02682
E3	gru_cnn_stem	99,27%	0,992663	0,02744
E3	lstm_overlap_patch	98,98%	0,989735	0,04211
E3	gru_overlap_patch	99,12%	0,991074	0,03837

LSTM dùng CNN-stem đạt accuracy test cao nhất quan sát được là 99,37% (63 lỗi), hơn CNN E1 0,10 điểm phần trăm và hai bản MSA CNN-stem 0,09 điểm phần trăm, tương ứng 10 và 9 ảnh trên 10.000 ảnh test. Hai baseline E1 được dùng lại trong Chương 4 nhưng không đếm thêm vào tổng 19 cấu hình. Các checkpoint đều được chọn theo validation; bảng test mô tả kết quả đã chạy, không dùng để chọn lại hyperparameter. Một seed chưa cho phép khẳng định mức chênh có ý nghĩa thống kê hay một họ kiến trúc luôn tốt hơn [2]–[6].

5.2. Hiệu năng so với chi phí mô hình

Bảng 14. Số tham số và thời gian huấn luyện cộng xác thực

Bài	Mã thí nghiệm	Tham số huấn luyện	Thời gian train + val
E1	softmax	7.850	718,3 s
E1	mlp	235.146	760,6 s
E1	cnn	421.642	770,0 s
E2	custom_patch	38.602	738,8 s
E2	pytorch_patch	38.602	733,0 s
E2	custom_row	38.026	750,6 s
E2	pytorch_row	38.026	751,3 s
E2	custom_cnn_stem	56.330	763,6 s
E2	pytorch_cnn_stem	56.330	827,2 s
E2	custom_overlap_patch	41.674	739,7 s
E2	pytorch_overlap_patch	41.674	737,4 s
E3	lstm_row	82.186	723,7 s
E3	gru_row	61.962	699,9 s
E3	lstm_patch	92.938	703,9 s
E3	gru_patch	70.026	688,0 s
E3	lstm_cnn_stem	119.434	743,1 s
E3	gru_cnn_stem	94.602	756,5 s
E3	lstm_overlap_patch	92.938	755,0 s
E3	gru_overlap_patch	70.026	749,3 s

E2 CNN-stem có 56.330 tham số, khoảng 13,36% CNN E1; LSTM CNN-stem E3 có 119.434, khoảng 28,33% CNN E1. Thời gian train + validation của E1 khoảng 718–770 giây, E2 khoảng 733–827 giây và E3 khoảng 688–756 giây. E2/E3 ghi nhận cùng loại GPU Tesla P100-PCIE-16GB và PyTorch 2.2.1+cu118, nhưng chưa kiểm soát tải cạnh tranh; E1 thiếu metadata GPU. Các số đo này không phải độ trễ suy luận và không đủ cho kết luận tổng quát về tốc độ [2]–[5].

5.3. Diễn giải kết quả theo giả thuyết kiến trúc

Softmax ánh xạ trực tiếp pixel sang lớp nên thiếu khả năng học các tổ hợp nét phi tuyến; MLP khắc phục phần nào bằng tầng ẩn, trong khi CNN có phép tích chập chia sẻ trọng số và cơ chế tổng hợp cục bộ. Trên MNIST, điều này phù hợp với quan sát

CNN có độ chính xác cao hơn, nhưng vẫn là diễn giải theo inductive bias của kiến trúc, không phải thí nghiệm ablation đã tách riêng từng thành phần [7], [8].

CNN-stem cho kết quả tốt nhất trong từng bản MSA và từng họ RNN ở các lượt đã chạy. Điều này phù hợp với ích lợi của đặc trưng không gian cục bộ, song CNN-stem bổ sung hai lớp tích chập và thay số tham số. Ở E3, patch chồng lấn hơn patch rời 0,78 điểm phần trăm với LSTM và 0,76 với GRU, trong khi giữ cùng chiều đặc trưng và số tham số tương ứng; số bước, mức chồng lấn và chi phí tính toán đồng thời thay đổi. E2 còn có projection, CLS và nhúng vị trí, còn E3 dùng pixel thô cho hàng/patch và trạng thái ẩn cuối để phân loại. Do đó so sánh E2 với E3 là so các cấu hình hoàn chỉnh, chưa tách riêng hiệu ứng attention so với hồi quy [3]–[5].

5.4. Hạn chế và cải tiến thực nghiệm

Mỗi cấu hình chỉ có một seed huấn luyện, chưa đủ đánh giá độ ổn định giữa các lượt. Cả ba bài cùng 100 epoch/lô 256 nhưng khác kiến trúc, dropout và cách tiêu thụ RNG; cùng siêu tham số chung không có nghĩa từng mô hình đã được tối ưu riêng. E2/E3 có metadata môi trường, nhưng chưa đo bộ nhớ hoặc độ trễ suy luận; E1 thiếu thông tin GPU. Kiểm chứng MSA là kiểm tra số học hữu hạn. Order probe chỉ đo độ nhạy thứ tự dưới sự thay đổi phân phối, không chứng minh phụ thuộc dài hạn [2]–[6].

Hướng kiểm chứng tiếp theo là chạy thêm các lượt với nhiều seed huấn luyện trên cùng split, đo thời gian và suy luận trên cùng thiết bị, giữ ngân sách tương đương và thử dữ liệu khó hơn. Các lượt nhiều seed cần giữ nguyên seed chia dữ liệu; checkpoint vẫn được chọn bằng validation trước khi đánh giá test.

KẾT LUẬN

Báo cáo tổng hợp 19 cấu hình độc lập và 1.900 bản ghi epoch, được đánh giá trong cùng quy trình 100 epoch/lô 256. E1 đạt accuracy test 92,59% (Softmax), 98,63% (MLP), 99,27% (CNN). E2 hoàn tất tám cấu hình và tám trường hợp kiểm chứng số học; CNN-stem đạt 99,28% ở cả hai bản MSA. E3 hoàn tất tám LSTM/GRU, dùng lại hai baseline E1; LSTM CNN-stem đạt 99,37% với 119.434 tham số, GRU CNN-stem đạt 99,27% với 94.602 tham số [2]–[6].

Kết quả cho thấy cách biểu diễn ảnh gắn với chất lượng dự đoán và chi phí mô hình trong thiết kế đã xét. E2 phân biệt được việc kiểm chứng hai hiện thực cùng MSA với việc so sánh các lượt huấn luyện. E3 đối chiếu LSTM/GRU trên từng biểu diễn và dùng baseline E1 nhất quán; thử đảo/xáo token bổ sung bằng chứng về độ nhạy thứ tự không gian. Những quan sát này chưa chứng minh một kiến trúc ưu việt trên dữ liệu khác hoặc có năng lực nhớ dài hạn. Kiểm chứng tiếp theo cần nhiều seed và điều kiện đo phần cứng được kiểm soát.

TÀI LIỆU THAM KHẢO

[1] Trường Đại học Bách khoa – ĐHQG TP.HCM. Bài tập E1–E3, học phần CO5085, Học kỳ 261. Nguồn: Exercise/exercise-vne.pdf, 6 trang.

[2] Mã nguồn trong repo: common/data.py, common/engine.py, common/plots.py, common/utils.py, E1/models.py và E1/train.py. E1 mới dùng cùng kiến trúc và training loop, thay tham số chạy thành 100 epoch, batch size 256.

[3] Mã nguồn E2/models.py, E2/train.py, E2/verify_msa.py và E2/comparison.py. Hash SHA-256 của mã nguồn được lưu trong E2/results/tokenization_msa_v2/metrics.json; khớp repo sau chuẩn hóa xuống dòng LF.

[4] Mã nguồn E3/models.py, E3/train.py, E3/baselines.py và E3/comparison.py. Hash nguồn lưu tại E3/results/sequence_v2/metrics.json; khớp repo sau chuẩn hóa xuống dòng LF. E3/smoke_test.py kiểm tra pipeline bằng dữ liệu tổng hợp, không tạo số liệu MNIST trong báo cáo.

[5] Metrics và summary tại E1/results/epochs100_batch256_seed42/, E2/results/tokenization_msa_v2/, E3/results/sequence_v2/. E2/E3 có CSV dự đoán và ma trận nhầm lẫn; E2 có verification.json. E3 có baselines_e1.json ghi nguồn baseline và comparison_order_probe.csv ghi độ nhạy trên validation.

[6] Các history_*.csv tại ba thư mục trong [5]: E1 có 300, E2 có 800, E3 có 800 bản ghi epoch; tổng 1.900 bản ghi của 19 cấu hình độc lập. History MLP/CNN E1 chỉ tính một lần.

[7] Slide học phần: 02-foundation/05-Losses.pdf (tr. vật lý 21–23), 02-foundation/08-MLP.pdf (tr. 7, 11–16), bản trích 09_TRICH_XUAT_SLIDE_LIEN_QUAN.txt.

[8] Slide học phần: 03-cnn/01-Conv2d-Overview.pdf (tr. vật lý 4–5, 39), 03-cnn/06-Pool-Dropout.pdf (tr. 4–5), bản trích 09.

[9] Slide học phần: 05-transformers/03-Model-Families-BERT-GPT-T5-ViT.pdf (tr. vật lý 29–32), bản trích 09.

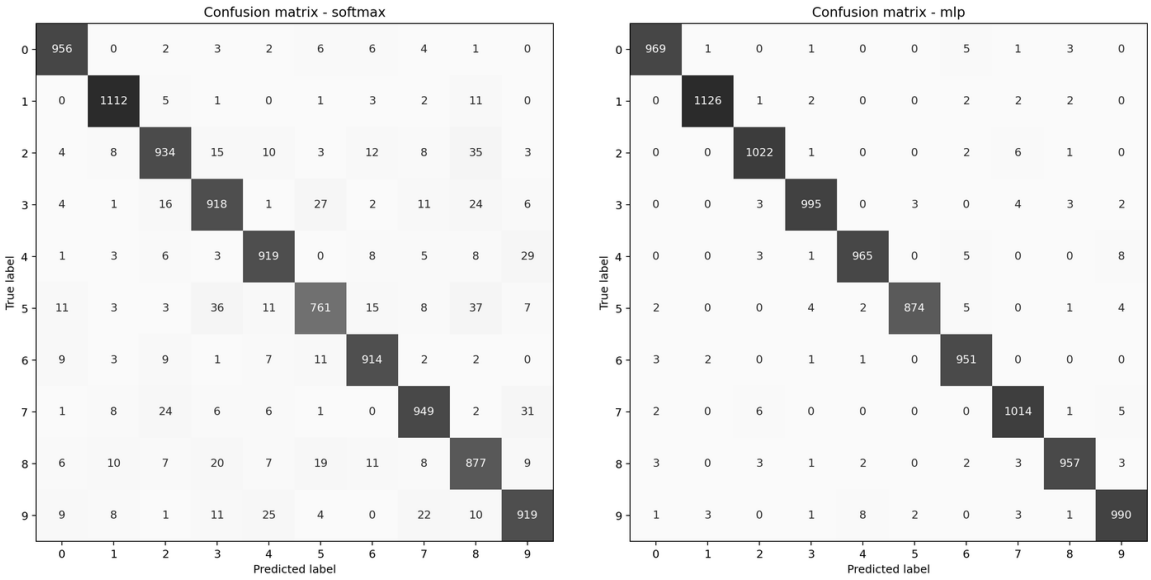
[10] Slide học phần: 05-transformers/02-Attention-Transformer-Core.pdf (tr. vật lý 11–12, 17–23), bản trích 09.

[11] Slide học phần: 04-rnn/RNN-Summary.pdf (tr. vật lý 24–34, 37–39), bản trích 09.

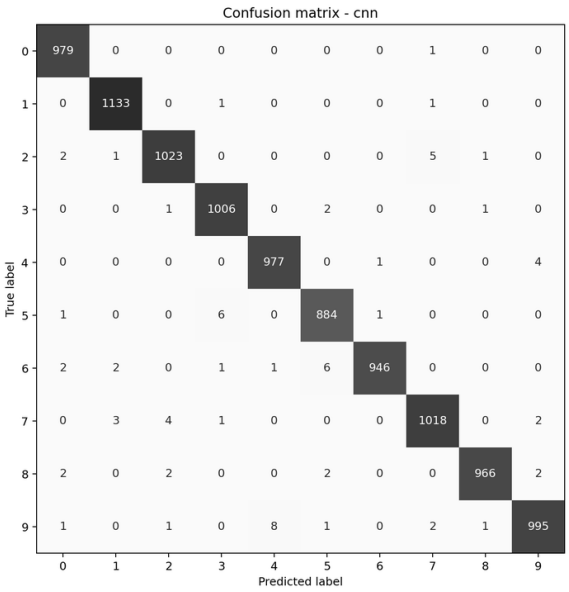
Nguồn hình: E1, E2, E3 lần lượt dùng đúng ba thư mục kết quả trong [5], [6]; không dùng lại hình của các lượt 20 epoch/lô 16.

PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN

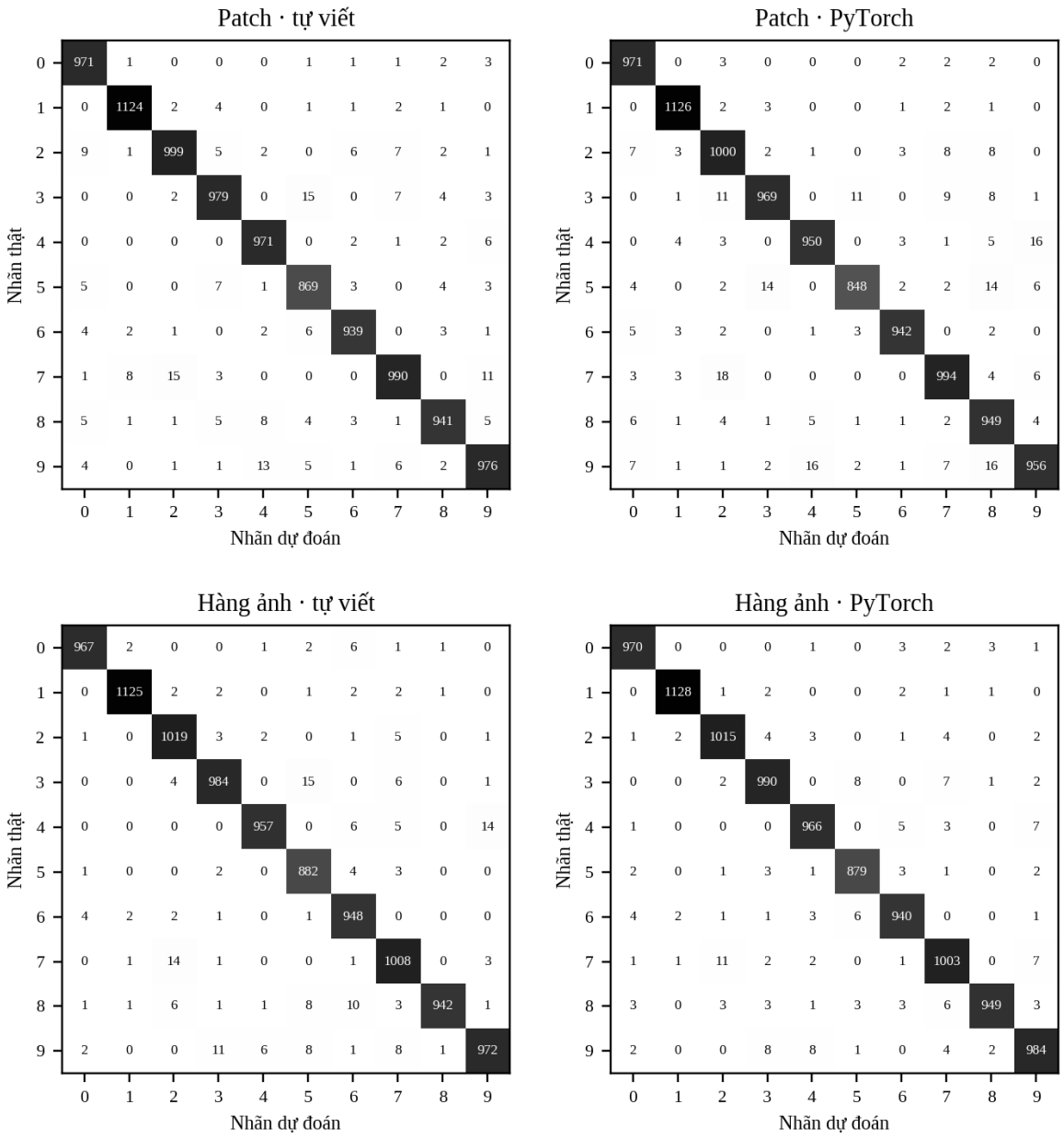
Các ma trận thuộc bộ 100 epoch/lô 256 của cả ba bài. Hàng là nhãn thật, cột là nhãn dự đoán. Ma trận E2/E3 ở đây được dựng từ 160.000 dự đoán từng ảnh trong CSV 18. MLP/CNN dùng làm baseline E3 chính là các mô hình E1 ở Hình 9–10, không có lượt huấn luyện baseline mới. Các hình dùng thang xám để thuận tiện đọc và in.



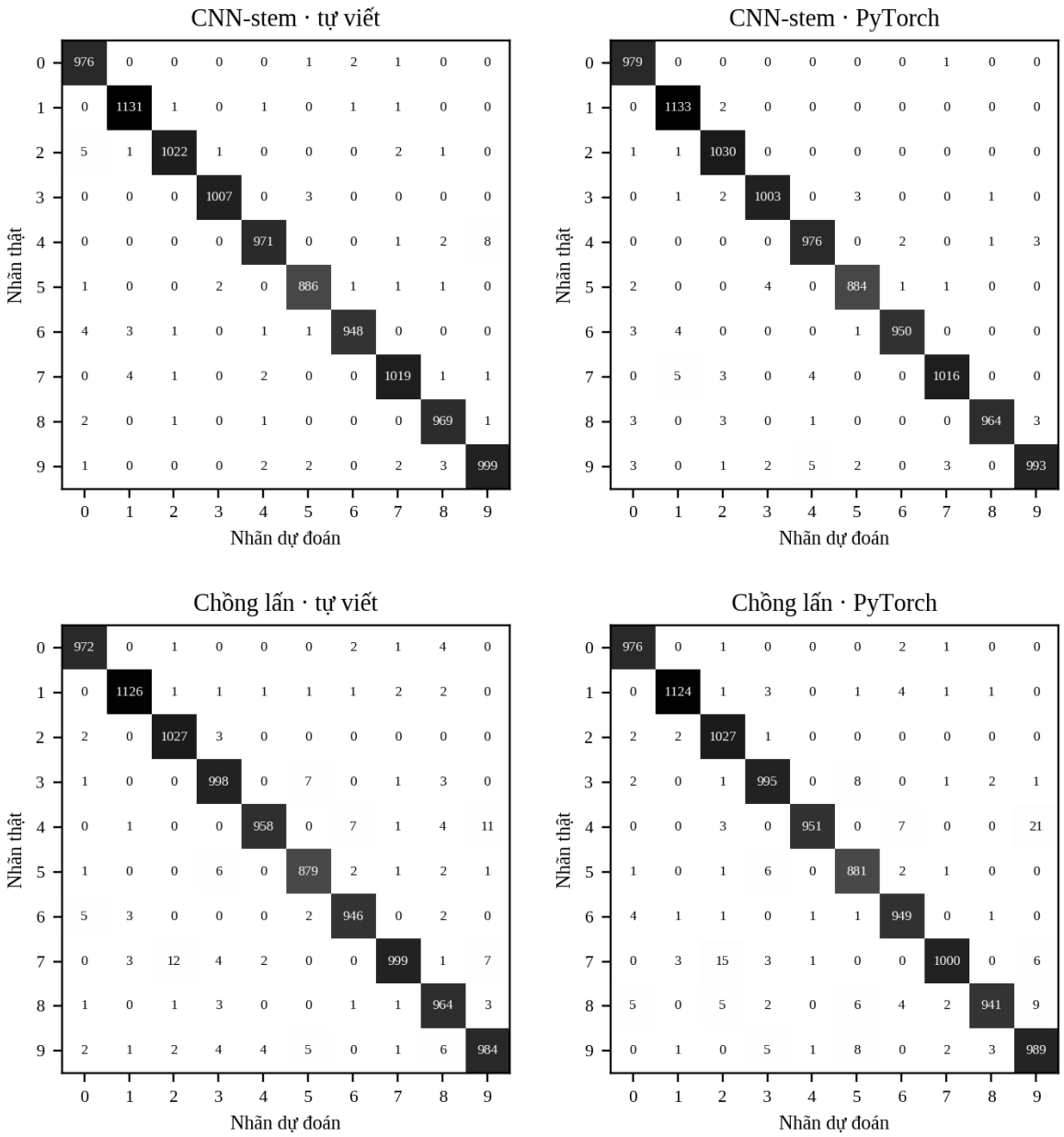
Hình 9. Ma trận E1: Softmax (trái) và MLP (phải), 100 epoch/lô 256.



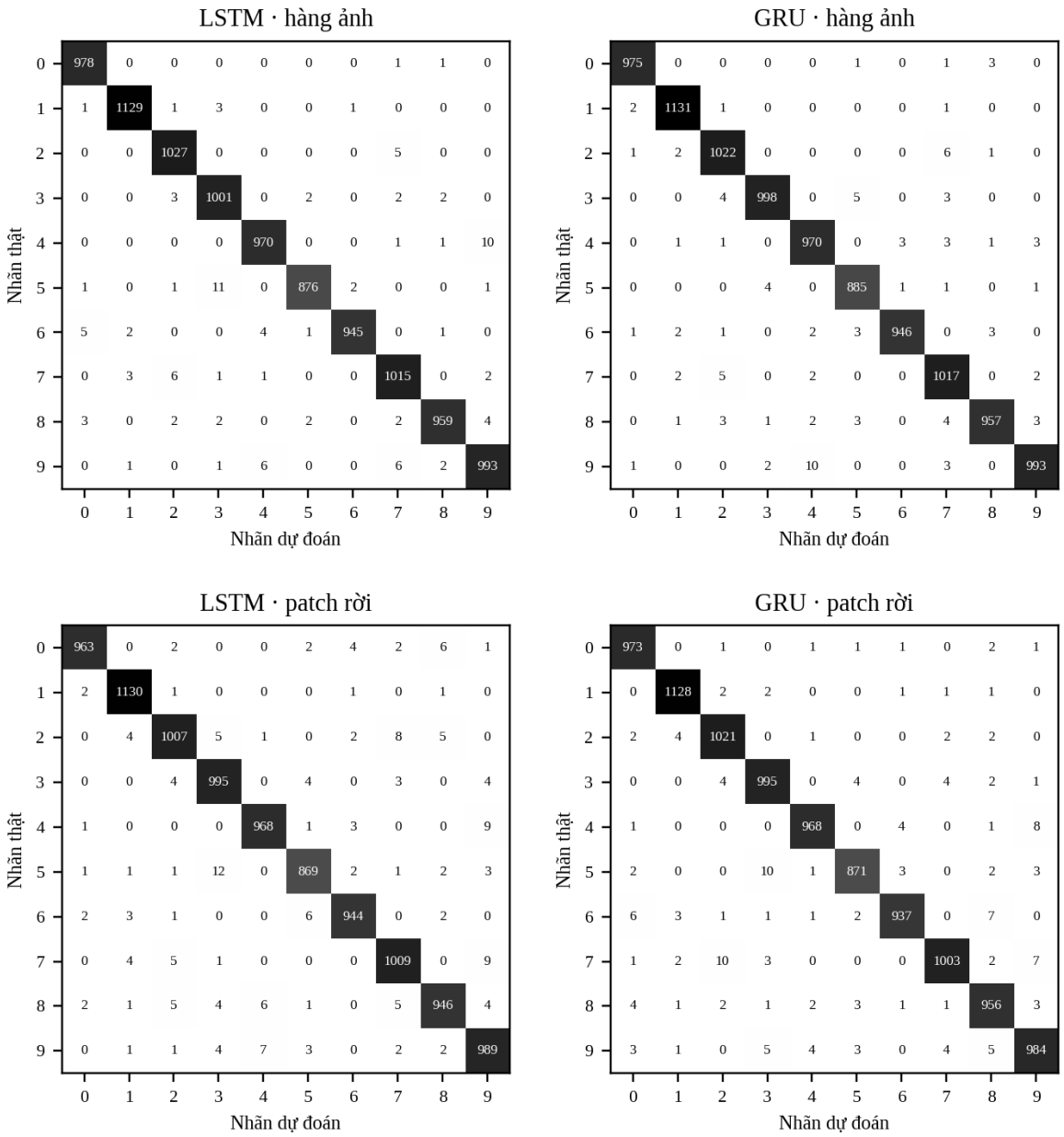
Hình 10. Ma trận E1: CNN, 100 epoch/lô 256.



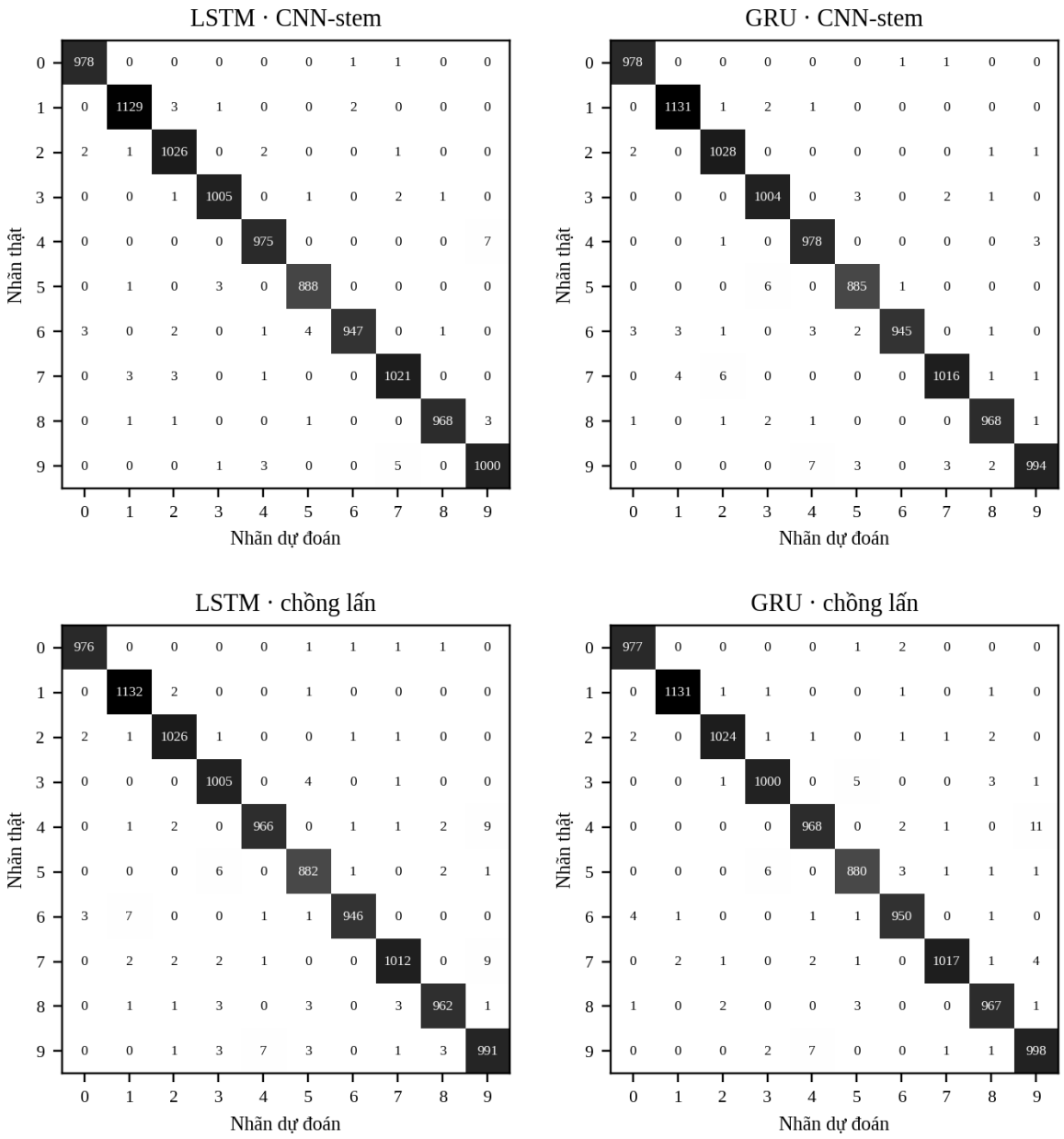
Hình 11. Ma trận E2: patch (hàng trên) và hàng ảnh (hàng dưới); tự viết bên trái, PyTorch bên phải.



Hình 12. Ma trận E2: CNN-stem (hàng trên) và patch chồng lấn (hàng dưới); tự viết bên trái, PyTorch bên phải.



Hình 13. Ma trận E3: hàng ảnh (hàng trên), patch rời (hàng dưới); LSTM bên trái, GRU bên phải.



Hình 14. Ma trận E3: CNN-stem (hàng trên), patch chồng lấn (hàng dưới); LSTM bên trái, GRU bên phải.

PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI

B.1. Lệnh gợi ý tái lập theo cấu hình đã lưu

Các lệnh dưới đây mô tả cấu hình đã chạy: 100 epoch/lô 256, split seed 42. Khi tái lập, dùng thư mục đầu ra mới; E2/E3 không hỗ trợ ghi đè hoặc resume. E2 tự kiểm chứng MSA trước huấn luyện; E3 kiểm tra nguồn baseline E1. Đổi --train-seed khi chạy lặp E2/E3 nhưng giữ --seed 42 để cố định split. Chỉ đổi seed E1 sẽ đồng thời đổi split theo giao diện hiện tại [2]–[5].

Lệnh 1: python E1/train.py --models all --epochs 100 --batch-size 256 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --device cuda --output-dir E1/results/epochs100_batch256_seed42

Lệnh 2: python E2/train.py --experiments all --epochs 100 --batch-size 256 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --train-seed 42 --device cuda --output-dir E2/results/tokenization_msa_v2

Lệnh 3: CUDA_VISIBLE_DEVICES=1 python E3/train.py --experiments all --epochs 100 --batch-size 256 --device cuda --progress --probe-order --output-dir E3/results/sequence_v2

B.2. Các điểm cần xác thực trong bước hoàn thiện

E2/E3 ghi nhận Python 3.10.13, PyTorch 2.2.1+cu118, torchvision 0.17.1+cu118, CUDA 11.8, Tesla P100-PCIE-16GB và hash nguồn/split. Tám checkpoint E2 và tám checkpoint E3 được giữ tại thư mục nguồn của từng lượt chạy, không đưa lên Git. Checkpoint E3 đã kiểm tra nạp được, khớp kiến trúc, số tham số và best epoch; đây không phải chạy lại toàn bộ test trên máy rà soát. E1 chưa kèm checkpoint hoặc dự đoán từng ảnh. Hash history baseline giữa Windows/Linux chỉ khác xuống dòng CRLF/LF; nội dung và chỉ số khớp. Các kết quả lịch sử tại thư mục cha được giữ để truy vết, không đưa vào bảng chính. Điểm cần nhóm kiểm tra tại E1: mục 2.4 của bản thảo 03 ghi CNN nhầm 9→4 ở 10 ảnh, trong khi ma trận E1 ở hình 13 gốc (Hình 10 của báo cáo) cho thấy 8 ảnh. Bản review đọc 8 theo ma trận, chưa kiểm chứng độc lập từ dự đoán từng ảnh E1 vì dữ liệu này chưa được cung cấp.

B.3. Khai báo sử dụng công cụ AI

Trong quá trình soạn và cập nhật báo cáo, công cụ AI hỗ trợ tổ chức tài liệu, đối chiếu mã nguồn với số liệu, diễn giải ban đầu, cập nhật bảng/hình và biên tập bản Word để nhóm xem xét. Đây là hỗ trợ soạn thảo và kiểm tra nội dung, không chỉ kiểm tra chính tả. Theo quy định liêm chính học thuật của học phần, người học chịu trách nhiệm kiểm tra, chỉnh lý, xác nhận kết quả, giải thích mã nguồn và bảo vệ các nhận định trước khi nộp bài [1]. Nhóm cần xác nhận khai báo này phản ánh đúng mức hỗ trợ thực tế trước khi nộp.